

CoralOpt

Coral Reef Conservation Optimizer

Project Report

End-to-End Data Engineering & Optimization Pipeline

Domain: Data Engineering, Optimization

Year: 2026

Status: Completed

Stack: Kafka · Spark · Airflow · Python

Contents

1	Project Overview	2
1.1	Background	2
1.2	Why This Project?	2
1.3	Objectives	2
2	Methodology	2
2.1	Overall Architecture	2
2.2	Approach	3
3	Technical Components	3
3.1	Kafka Producer	3
3.2	Spark Processor	4
3.3	Airflow DAG	5
3.4	LP Solver — Linear Programming	5
3.5	Standardizer	6
3.6	Streamlit Dashboard	7
4	Results	7
4.1	Component Status	7
5	Tech Stack	8
6	CV / Portfolio Highlights	8
7	Future Work	8

1. Project Overview

1.1 Background

Coral reefs are among the most biodiverse ecosystems on Earth, covering less than 1% of the ocean floor yet supporting more than 25% of all marine species. However, coral bleaching driven by climate change and rising sea surface temperatures now poses a severe threat to reefs worldwide.

According to NOAA Coral Reef Watch, sustained sea surface temperatures (SST) above 29°C are the primary driver of mass bleaching events. Real-time SST monitoring and optimal allocation of limited conservation budgets are therefore critical challenges for marine conservation organizations.

1.2 Why This Project?

- **Real-world relevance:** Uses live data from the NOAA ERDDAP API, reflecting the actual status of reef zones around the globe.
- **Multi-domain integration:** Combines Data Engineering (Kafka, Spark, Airflow), mathematical optimization (Linear Programming), and Data Science (risk scoring).
- **DE/DS skill demonstration:** End-to-end pipeline from raw ingestion to dashboard, closely mirroring real production workflows.
- **Social impact:** Supports data-driven conservation decision-making, replacing intuition-based approaches.

1.3 Objectives

1. Build a real-time SST data ingestion pipeline from NOAA via Kafka.
2. Process data and compute risk scores for each reef zone using PySpark.
3. Orchestrate the full pipeline automatically every day with Airflow.
4. Apply Linear Programming to optimally allocate conservation budgets.
5. Visualize results through an interactive Streamlit dashboard.

2. Methodology

2.1 Overall Architecture

The project follows a **Lambda-inspired pipeline** architecture, organized into five distinct layers:

Pipeline Architecture

```
NOAA API → Kafka → Spark → Airflow → LP  
Solver → Streamlit
```

2.2 Approach

1. **Data Ingestion:** A Kafka Producer collects SST readings from the NOAA ERD-DAP API every 60 seconds. When the API is unavailable, the system automatically falls back to mock data, ensuring the pipeline never stalls.
2. **Stream Processing:** PySpark reads all messages from the Kafka topic, parses JSON, computes a composite risk score via a User Defined Function (UDF), aggregates by reef zone, and writes results to Parquet.
3. **Orchestration:** An Airflow DAG schedules the full pipeline at 6 AM daily in the following order: Kafka → Spark → Verify → LP Solver.
4. **Optimization:** The LP Solver reads risk scores from the Parquet output, formulates a linear program, solves it using the Simplex algorithm (Dantzig's rule), and returns the optimal budget allocation plan.
5. **Visualization:** The Streamlit dashboard displays reef status and optimization results. Users can enter any budget and receive a real-time allocation plan without exposure to any LP terminology.

3. Technical Components

3.1 Kafka Producer

Apache Kafka

File: `Kafka_producer.py` **Broker:** Confluent Cloud (SASL_SSL) **Topic:** coral-raw-data

Kafka acts as the **message broker** in the pipeline, completely decoupling data collection from data processing. Because Mac M-series (ARM64) is incompatible with x86_64 Kafka Docker images, the project uses **Confluent Cloud free tier** — a more production-ready approach that eliminates local JVM compatibility issues.

Bleaching Risk Formula

$$\text{bleaching_risk} = \begin{cases} 0.0 & \text{if } SST \leq 29^{\circ}\text{C} \\ \frac{SST - 29}{2} & \text{if } 29^{\circ}\text{C} < SST < 31^{\circ}\text{C} \\ 1.0 & \text{if } SST \geq 31^{\circ}\text{C} \end{cases} \quad (1)$$

Message Schema

Each message published to Kafka has the following structure:

Listing 1: Kafka message schema

```
1 message = {
2     "reef_id"      : "CAR_01",
3     "name"         : "Caribbean Reef",
```

```

4      "lat"           : 15.0 ,
5      "lon"           : -63.0 ,
6      "sst_celsius"    : 26.89 ,
7      "bleaching_risk" : 0.0 ,
8      "data_source"    : "NOAA_ERDDAP", # or "mock"
9      "timestamp"      : "2026-05-17T12:34:56"
10   }

```

3.2 Spark Processor

Apache Spark

File: spark_processor.py **Version:** PySpark 4.1.1 **Kafka Connector:** spark-sql-kafka-0-10_2.13:4.0.0

PySpark reads data from Kafka, parses JSON against a fixed schema, computes a composite risk score via UDF, aggregates by reef zone, and saves results as Parquet.

Composite Risk Score Formula

$$\text{risk_score} = 0.4 \times \text{sst_factor} + 0.6 \times \text{bleaching_risk} \quad (2)$$

$$\text{sst_factor} = \text{clip}\left(\frac{SST - 27}{4}, 0, 1\right) \quad (3)$$

Table 1: Risk score component weights

Component	Weight	Rationale
SST factor	0.4	Indirect indicator — high temperature <i>may</i> trigger bleaching
Bleaching risk	0.6	Direct indicator — coral <i>is already</i> bleaching

Output

Spark saves aggregated results as Parquet at `./output/risk_scores/`:

Listing 2: Spark output — Risk Score per reef zone

```

+-----+-----+-----+-----+-----+
| reef_id | name                | avg_sst | avg_risk_score |
| avg_bleaching_risk |
+-----+-----+-----+-----+-----+
| CAR_01  | Caribbean Reef      | 26.886  | 0.0262         |
| 0.0     |                      |         |                 |
| GBR_01  | Gr. Barrier Reef North | 26.044  | 0.0            |
| 0.0     |                      |         |                 |

```

GBR_02	Gr. Barrier Reef South	25.842	0.0
0.0			
CRL_01	Coral Sea	26.01	0.0
0.0			
RED_01	Red Sea North	25.168	0.0
0.0			
+-----+-----+-----+-----+			

3.3 Airflow DAG

⚙️ Apache Airflow

File: coralopt_dag.py **DAG ID:** coralopt_pipeline **Schedule:** 0 6 *
* * (6 AM daily)

Airflow orchestrates the full pipeline in the following task order:

kafka_producer → spark_processor → check_results → lp_solver

Table 2: Airflow DAG task summary

Task	Avg Duration	Description
kafka_producer	~30s	Sends SST readings for 5 reef zones to Kafka
spark_processor	~4–5min	Processes messages, computes risk scores, saves Parquet
check_results	<5s	Verifies Parquet output exists before LP step
lp_solver	~1–2s	Optimally allocates default \$500k budget

3.4 LP Solver — Linear Programming

📊 Linear Programming

Files: standardizer.py, danzig_solver.py, bland_solver.py,
twophase_solver.py, lp_solver_pipeline.py

LP Formulation

$$\text{Maximize } \sum_{i=1}^n \text{risk_score}_i \cdot x_i \quad (4)$$

$$\text{subject to } \sum_{i=1}^n \text{cost}_i \cdot x_i \leq B \quad (\text{budget constraint}) \quad (5)$$

$$0 \leq x_i \leq \text{area}_i \quad \forall i \quad (\text{area constraints}) \quad (6)$$

$$(7)$$

where x_i is the area (km²) restored in reef zone i , and B is the total conservation budget.

Input Data

Table 3: Parameters per reef zone

Reef ID	Zone Name	Cost (USD/km ²)	Area (km ²)
GBR_01	Great Barrier Reef North	15,000	350
GBR_02	Great Barrier Reef South	12,000	280
CRL_01	Coral Sea	18,000	120
RED_01	Red Sea North	20,000	90
CAR_01	Caribbean Reef	16,000	150

Simplex Algorithms Implemented

Table 4: Comparison of Simplex algorithms

Algorithm	Entering Variable Rule	Key Property
Dantzig	Most negative reduced cost	Fast convergence in practice
Bland	Smallest index with negative cost	Guaranteed finite termination (anti-cycling)
Two-Phase	Dantzig or Bland (auto-selected)	Handles $\geq$ and $=$ constraints

LP Result with \$500,000 Budget

Listing 3: LP Solver output

```

Status      : Optimal
Budget      : $500,000
Used        : $500,000 (100.0%)
Total impact: 0.8156

Allocation:
reef_id  name          area_restored  cost_used  impact
CAR_01   Caribbean Reef  31.25 km2     $500,000   0.8156
CRL_01   Coral Sea          0.00 km2     $0         0.0000
GBR_01   GBR North          0.00 km2     $0         0.0000

```

Caribbean Reef receives the full budget because it has the highest risk score (0.0262) at a competitive restoration cost of \$16,000/km².

3.5 Standardizer

The `standardizer.py` module converts a general LP problem into standard form required by the Simplex solvers:

1. **Max** $\rightarrow$ **Min**: Negate the objective coefficients for maximization problems.

2. **Variable substitution:** $x_j \leq 0$ replaced by $x'_j = -x_j$; free variables split into $x'_j - x''_j$.
3. **Non-negative RHS:** Rows with $b_i < 0$ are multiplied by -1 and constraint direction is flipped.
4. **Auxiliary variables:** Slack added for $\leq$, surplus for $\geq$, none for $=$.

3.6 Streamlit Dashboard

Streamlit Dashboard

File: app.py **URL:** localhost:8501

The dashboard consists of two tabs:

- **Tab 1 — Reef Status:** Displays risk scores, SST readings, and status labels per zone. Includes summary metrics and dual bar charts (risk and temperature).
- **Tab 2 — Budget Planning:** User enters a budget $\rightarrow$ the system calls `run_lp_solver(budget)` $\rightarrow$ returns an optimal allocation table, pie chart, and bar chart. No LP terminology is exposed to the user.

4. Results

Results Achieved

- Complete end-to-end pipeline: NOAA $\rightarrow$ Kafka $\rightarrow$ Spark $\rightarrow$ Airflow $\rightarrow$ LP Solver $\rightarrow$ Streamlit
- Kafka Producer connected to Confluent Cloud, streaming data every 60s with automatic mock fallback
- Spark Processor successfully computed risk scores and saved Parquet output
- Airflow DAG completed in ~ 7 minutes, scheduled automatically at 6 AM daily
- LP Solver found optimal solution: \$500k $\rightarrow$ Caribbean Reef, impact = 0.8156
- Streamlit dashboard is user-friendly with no LP terminology exposed

4.1 Component Status

Table 5: Project component status

Component	Technology	Status
Data Ingestion	Kafka (Confluent Cloud)	Complete
Stream Processing	PySpark 4.1.1	Complete
Orchestration	Apache Airflow	Complete
Optimization	LP / Simplex (from scratch)	Complete
Visualization	Streamlit	Complete

5. Tech Stack

Table 6: Technologies used in the project

Technology	Version	Role
Python	3.12	Primary language
kafka-python	2.3.1	Kafka Producer client
Confluent Cloud	Free tier	Managed Kafka broker (ARM-compatible)
PySpark	4.1.1	Stream processing and risk scoring
Apache Airflow	Latest	Pipeline orchestration
Azul Zulu JDK	17.66+19	JVM for Spark and Airflow (ARM64)
NumPy	—	Matrix operations in LP solver
Pandas	—	DataFrame handling for results
Streamlit	—	Web dashboard
Matplotlib	—	Charts and visualizations
NOAA ERDDAP	—	Real-time SST data source

6. CV / Portfolio Highlights

- End-to-end pipeline:** From raw API data to a production-ready dashboard — no steps skipped.
- Production-aware decisions:** Chose Confluent Cloud over local Docker due to ARM incompatibility — demonstrates practical constraint handling.
- Custom algorithm implementation:** Built Dantzig, Bland, and Two-Phase Simplex from scratch using NumPy — no third-party LP libraries.
- Fault tolerance:** Automatic mock data fallback when NOAA API times out — pipeline never crashes.
- Domain integration:** Combined oceanography (NOAA Coral Reef Watch bleaching formula) with data engineering and mathematical optimization.
- Separation of concerns:** Each file has a single, clearly defined responsibility — easy to maintain and extend.

7. Future Work

- Stabilize NOAA API connection to replace mock data with live readings
- Expand monitoring to more reef zones beyond the current 5
- Deploy Streamlit to the cloud (Streamlit Community Cloud or AWS)
- Add an alert system via email or Slack when risk scores exceed critical thresholds

- Integrate additional ocean health indicators: pH, dissolved oxygen, satellite coral coverage imagery
- Benchmark Dantzig, Bland, and Two-Phase Simplex on larger-scale LP problems